

Testkonzept für das Open-Source-Projekt „lazygit„

Belegarbeit im Studiengang Informatik (B.Sc.)

**Grundlagen des Softwaretestens
Wintersemester 2025/26**

Dozent: Prof. Dr. Matthias Längrich

Hochschule Zittau/Görlitz
Fakultät Elektrotechnik und Informatik

**Verfasser: Konrad Miosga
Matrikelnummer: 1056710
E-Mail-Adresse: konrad.miosga@stud.hszg.de**

Datum der Abgabe: 20.01.2026

Inhaltsverzeichnis

1. Einleitung	1
1.1. Projektkontext	1
1.2. Zielsetzung und Methodik	1
2. Theoretische Grundlagen	1
3. Analyse der vorhandenen Teststrategie	3
3.1. Unit-Tests	4
3.2. Table-Driven Tests	5
3.3. Integrationstests	6
4. Continuous Integration Pipeline	7
4.1. Job-Struktur und Parallelisierung	8
4.1.1. Coverage-Aggregation	9
4.1.2. Fail-Safes und Quality Gates	9
4.2. Code-Coverage-Analyse und Metriken	10
4.2.1. Coverage-Erfassung in Unit-Tests	10
4.2.2. Coverage-Erfassung in Integrationstests	10
4.2.3. Coverage-Merging	10
4.2.4. Coverage-Metriken nach Komponenten	10
4.2.5. Testmetriken	11
5. Testumgebung und Werkzeuge	11
5.1. Verwendete Tools	11
5.2. Testumgebungen	11
5.3. Docker-Container	12
6. Entwurf eigener Testfälle	12
6.1. Identifikation von Testlücken	12
6.2. Testfalldesign und Methodik	12
6.3. Implementierung	13
6.4. Testergebnisse	13
7. Testauswertung und Metriken	14
7.1. Empfehlungen und Fazit	14
7.1.1. Bewertung der Teststrategie	14
7.1.2. Handlungsempfehlungen	16
7.1.3. Fazit	17

Quellen	19
---------------	----

1. Einleitung

1.1. Projektkontext

In dieser Belegarbeit wird das Testkonzept des Open-Source-Projekts **lazygit** analysiert, bewertet und durch eigene Testfälle ergänzt. Lazygit ist eine in Go entwickelte Terminal-UI für Git-Kommandos, die komplexe Git-Operationen durch eine intuitive, tastaturgesteuerte Benutzeroberfläche vereinfacht. Das Projekt wurde von Jesse Duffield initiiert, wird seit 2018 aktiv entwickelt und gehört mit über 50.000 GitHub-Stars zu den populärsten Git-Tools im Terminal-Bereich.

Die Architektur folgt dem MVC-Muster mit klarer Trennung zwischen UI-Komponenten, Business-Logik und Git-Operationen. Diese Struktur erleichtert das isolierte Testen einzelner Komponenten erheblich.

1.2. Zielsetzung und Methodik

Diese Arbeit verfolgt mehrere Ziele: Analyse der bestehenden Teststrategie mit ihren Stärken und Schwächen, Identifikation von Testlücken durch Coverage-Analysen und Code-Reviews, praktische Implementierung neuer Testfälle sowie Ableitung konkreter Handlungsempfehlungen.

Die Bearbeitung erfolgte systematisch: Nach Literaturrecherche zu Testing-Grundlagen wurde die bestehende Testinfrastruktur analysiert. Coverage-Reports und manuelle Code-Reviews identifizierten Testlücken, die anschließend durch table-driven Tests geschlossen wurden. Die neuen Tests wurden in die CI-Pipeline integriert und auf mehreren Plattformen validiert.

2. Theoretische Grundlagen

Software-Testing ist ein zentraler Bestandteil der Qualitätssicherung in der Softwareentwicklung. Das oft zitierte Statement von Edsger W. Dijkstra bringt dabei eine grundlegende Eigenschaft des Testens prägnant auf den Punkt: Tests eignen sich hervorragend zum Aufdecken von Fehlern, sind jedoch ungeeignet, um die vollständige Fehlerfreiheit eines Programms nachzuweisen [1].

Diese Einschränkung ergibt sich aus der Natur des Software-Testings als Stichprobenverfahren. Da Programme in der Regel eine sehr große Menge möglicher Eingaben besitzen, kann im Rahmen von Tests nur eine endliche Teilmenge davon überprüft werden. Ein formaler Beweis der Korrektheit allein durch Tests ist daher prinzipiell nicht möglich.

Vor diesem Hintergrund kommt der systematischen und zielgerichteten Auswahl repräsentativer Testfälle eine zentrale Bedeutung zu. Ziel des Testens ist es nicht, Fehlerfreiheit zu garantieren, sondern mit begrenztem Aufwand eine möglichst hohe Wahrscheinlichkeit zur Entdeckung relevanter Defekte zu erreichen.

Der Testprozess lässt sich grundsätzlich in zwei zentrale Bereiche gliedern: die statische Analyse und das dynamische Testen [2, S. 4]. Die statische Analyse untersucht Softwareartefakte ohne deren Ausführung und zielt darauf ab, potenzielle Fehler, Regelverletzungen oder Qualitätsmängel frühzeitig im Entwicklungsprozess zu identifizieren [2, S. 43–45]. Zu den typischen Verfahren zählen Code-Reviews, der Einsatz von Lintern sowie statische Datenfluss- und Kontrollflussanalysen.

Demgegenüber steht das dynamische Testen, bei dem die Software mit konkreten Eingabedaten ausgeführt wird. Dadurch lassen sich insbesondere solche Fehler aufdecken, die erst zur Laufzeit auftreten, etwa Speicherlecks, Race Conditions oder fehlerhaftes Laufzeitverhalten. Dynamische Tests ermöglichen somit eine Überprüfung des tatsächlichen Systemverhaltens unter realistischen oder gezielt konstruierten Bedingungen [2, S. 39–43]. Der Schwerpunkt dieser Arbeit liegt auf dem dynamischen Testen und den zugehörigen Testverfahren.

Dynamisches Testen lässt sich grundsätzlich in Black-Box-Tests und White-Box-Tests unterteilen. Black-Box-Tests leiten Testfälle ausschließlich aus Spezifikationen und Anforderungen ab, ohne Kenntnis der internen Programmstruktur. Typische Verfahren sind hierbei die Äquivalenzklassenbildung sowie die Grenzwertanalyse. White-Box-Tests hingegen konstruieren Testfälle auf Basis der internen Programmstruktur mit dem Ziel, definierte Abdeckungskriterien wie Anweisungs- oder Pfadabdeckung zu erfüllen [3, S. 173–174]. In der praktischen Testpraxis werden beide Ansätze häufig kombiniert, um sowohl funktionale als auch strukturelle Aspekte der Software zu überprüfen.

Innerhalb der White-Box-Tests lassen sich verschiedene, aufeinander aufbauende Testverfahren unterscheiden. Unit-Tests überprüfen die kleinsten testbaren Einheiten eines Programms in Isolation. Abhängige Komponenten werden dabei typischerweise durch Mocks oder Stubs ersetzt, was schnelle, reproduzierbare und deterministische Tests ermöglicht [2, S. 371–372]. Eine häufig eingesetzte Ausprägung sind sogenannte Table-Driven Tests, bei denen mehrere Testfälle in Form von Datentabellen definiert und von einem generischen Testcode iterativ ausgeführt werden [4]. Dieses Vorgehen reduziert Redundanz und verbessert die Wartbarkeit der Tests.

Aufbauend auf den Unit-Tests folgen Integrationstests, die das Zusammenwirken mehrerer bereits getesteter Module überprüfen. Ziel ist es insbesondere, Fehler an Schnittstellen sowie Probleme im Zusammenspiel der Komponenten aufzudecken [2, S. 372–376]. Im Gegensatz zu Unit-Tests kommen hierbei überwiegend reale Komponenten statt isolierender Mocks zum Einsatz, wodurch eine höhere Aussagekraft hinsichtlich der Gesamtfunktionalität des Systems erreicht wird.

3. Analyse der vorhandenen Teststrategie

Lazygit verfolgt eine zweistufige Teststrategie, die aus Unit-Tests und einem sehr umfassenden Satz an Integrationstests besteht. Die Kernphilosophie des Projekts legt dabei einen deutlich größeren Wert auf Integrationstests, da diese das Verhalten der Anwendung aus der Perspektive des Endbenutzers am besten abbilden und somit als wertvoller für die Qualitätssicherung angesehen werden.

Unit-Tests bilden die unterste Ebene der Teststrategie. Sie befinden sich direkt neben dem zu testenden Code im pkg/-Verzeichnis und sind darauf ausgelegt, schnell und isoliert zu laufen. Um Funktionen mit einer Vielzahl von Eingaben und Grenzfällen zu prüfen, folgt das Projekt konsequent dem Go-Standard der Table-Driven Tests. Dabei wird eine „Tabelle“ (eine Slice von Structs) mit verschiedenen Testszenarien definiert, die dann in einer Schleife abgearbeitet werden. Jeder Fall wird als benannter Sub-Test ausgeführt, was die Tests übersichtlich, wartbar und leicht erweiterbar macht. Abhängigkeiten zur Außenwelt wie dem Dateisystem oder echten Git-Repositories werden durch Mocks ersetzt.

Der Schwerpunkt der Qualitätssicherung liegt auf den Integrationstests. Für diese wurde ein eigenes Test-Framework geschaffen. Anstatt einzelne Funktionen zu testen, simulieren diese Tests die Anwendung als Ganzes. Für jeden Test wird ein echtes Git-Repository in einem definierten Zustand erstellt. Anschließend steuert ein „Test-Treiber“ programmgesteuert die textbasierte Benutzeroberfläche (TUI), simuliert Tastendrücke und überprüft den Zustand der Oberfläche sowie die Ergebnisse von Git-Befehlen. Dieses Vorgehen stellt sicher, dass komplexe Arbeitsabläufe wie interaktive Rebases, Cherry-Picking oder das Beheben von Merge-Konflikten korrekt funktionieren.

Das Framework bietet zwei Ausführungsmodi: einen CLI-Modus für die automatisierte Ausführung (z.B. in CI/CD-Pipelines) und einen interaktiven TUI-Modus, der es Entwicklern erlaubt, Tests live in einer speziellen UI auszuwählen, auszuführen und zu debuggen. Dieser interaktive Modus erleichtert die Erstellung und Wartung der komplexen Tests erheblich. Das Hinzufügen neuer Tests ist ein klar definierter Prozess, bei dem nach dem Erstellen der Testdatei ein Codegenerierungs-Befehl ausgeführt wird, um den neuen Test in die globale Testliste zu integrieren.

3.1. Unit-Tests

Die Unit-Tests basieren auf einem Mock-Framework. Der `FakeCmdObjRunner` ist die zentrale Komponente und ermöglicht Thread-sichere, deterministische Tests ohne tatsächliche Git-Ausführung.

```
type FakeCmdObjRunner struct {  
    t *testing.T  
    expectedCmds []CmdObjMatcher  
    invokedCmdIndexes []int  
    mutex sync.Mutex  
}
```

Der FakeRunner verwaltet eine Liste erwarteter Commands (`expectedCmds`) und protokolliert, welche bereits ausgeführt wurden (`invokedCmdIndexes`). Der Mutex gewährleistet Thread-Safety bei paralleler Testausführung, was wichtig ist, da Go-Tests mit `t.Parallel()` konkurrent laufen können.

Das `CmdObjMatcher`-Interface erlaubt flexible Command-Matching:

```

type CmdObjMatcher struct {
    description string
    test func(*CmdObj) bool
    output string
    err error
}

```

Dies ermöglicht sowohl exaktes Matching (für spezifische Git-Befehle) als auch Pattern-basiertes Matching (für variable Parameter). Die test-Funktion entscheidet, ob ein Command zur Erwartung passt, während output und err die simulierte Antwort definieren.

Ein klassischer Unit-Test demonstriert den Einsatz:

```

func TestBranchNewBranch(t *testing.T) {
    runner := oscommands.NewFakeRunner(t).
        ExpectGitArgs([]string{"checkout", "-b", "test", "refs/heads/master"}, "", nil)
    instance := buildBranchCommands(commonDeps{runner: runner})

    assert.NoError(t, instance.New("test", "refs/heads/master"))
    runner.CheckForMissingCalls()
}

```

Die ExpectGitArgs-Methode registriert eine Erwartung für einen spezifischen Git-Befehl. CheckForMissingCalls() am Ende stellt sicher, dass alle erwarteten Befehle tatsächlich ausgeführt wurden – ein wichtiger Safeguard gegen unvollständige Tests.

3.2. Table-Driven Tests

Table-Driven Tests werden konsequent eingesetzt. Ein Beispiel aus branch_test.go zeigt die Struktur:

```

func TestBranchGetCommitDifferences(t *testing.T) {
    type scenario struct {
        testName      string
        runner         *oscommands.FakeCmdObjRunner
        expectedPushables string
        expectedPullables string
    }

    scenarios := []scenario{
        {
            "Can't retrieve pushable count",
            oscommands.NewFakeRunner(t).
                ExpectGitArgs([]string{"rev-list", "@{u}..HEAD", "--count"}, "",
errors.New("error")),
            "?", "?",
        },
        {
            "Retrieve pullable and pushable count",
            oscommands.NewFakeRunner(t).
                ExpectGitArgs([]string{"rev-list", "@{u}..HEAD", "--count"}, "1\n", nil).
                ExpectGitArgs([]string{"rev-list", "HEAD..@{u}", "--count"}, "2\n", nil),
            "1", "2",
        },
    }
}

```



```

    },
}

for _, s := range scenarios {
    t.Run(s.testName, func(t *testing.T) {
        instance := buildBranchCommands(commonDeps{runner: s.runner})
        pushables, pullables := instance.GetCommitDifferences("HEAD", "@{u}")
        assert.EqualValues(t, s.expectedPushables, pushables)
        assert.EqualValues(t, s.expectedPullables, pullables)
        s.runner.CheckForMissingCalls()
    })
}
}

```

Dieser Test validiert verschiedene Szenarien: normale Ausführung, Fehlerbehandlung bei der ersten Git-Operation und Fehlerbehandlung bei der zweiten Operation. Jedes Szenario definiert eigene Mock-Erwartungen, was präzise Kontrolle über Fehlerszenarien ermöglicht. Die Verwendung von `t.Run()` erstellt benannte Sub-Tests, was bei Fehlschlägen sofort zeigt, welches Szenario betroffen ist.

3.3. Integrationstests

Die Integrationstestsuite ist beeindruckend umfangreich mit über 450 Testdateien und etwa 28.000 Zeilen Code. Sie verwendet ein Domain-Specific Language (DSL)-Framework, das komplexe UI-Interaktionen lesbar beschreibt.

Ein Beispiel aus `branch/rebase.go` zeigt die DSL:

```

var Rebase = NewIntegrationTest(NewIntegrationTestArgs{
    Description: "Rebase onto another branch, deal with the conflicts.",
    SetupRepo: func(shell *Shell) {
        shared.MergeConflictsSetup(shell)
    },
    Run: func(t *TestDriver, keys config.KeybindingConfig) {
        t.Views().Commits().TopLines(
            Contains("first change"),
            Contains("original"),
        )

        t.Views().Branches().
            Focus().
            Lines(
                Contains("first-change-branch"),
                Contains("second-change-branch"),
            ).
            SelectNextItem().
            Press(keys.Branches.RebaseBranch)

        t.ExpectPopup().Menu().
            Title(Equals("Rebase 'first-change-branch'")).
            Select(Contains("Simple rebase")).
            Confirm()
    },
})

```

```
        t.Common().AcknowledgeConflicts()
    },
})
```

Diese Tests erstellen echte Git-Repositories mit dem `Shell-Helper`, simulieren Benutzerinteraktionen über `TestDriver` und validieren UI-Zustand durch Assertions wie `Contains()` und `Equals()`. Die Fluent API macht Tests selbstdokumentierend – man kann die User Journey direkt aus dem Code ablesen.

Die Integrationstests decken folgende Kategorien ab:

- **Branch-Operationen:** Checkout, Create, Delete, Rebase (über 100 Testdateien)
- **Commit-Operationen:** Amend, Revert, Cherry-Pick, Squash (über 120 Testdateien)
- **Interaktive Rebases:** Complex workflows mit 140+ Testdateien
- **Conflict-Handling:** Merge- und Rebase-Konflikte
- **File-Operations:** Staging, Unstaging, Discarding
- **Worktree-Management:** Multi-Worktree Szenarien
- **Custom Commands:** User-definierte Git-Operationen

Das Framework bietet spezialisierte Driver für verschiedene UI-Komponenten:

- `ViewDriver`: Interaktion mit Listen-Views (Branches, Commits, Files)
- `MenuDriver`: Navigation in Popup-Menüs
- `PromptDriver`: Eingabe in Text-Prompts
- `ConfirmationDriver`: Handling von Confirm-Dialogen
- `AlertDriver`: Validierung von Error-Messages

Diese Abstraktion trennt Testlogik von UI-Implementation, was Refactorings erleichtert. Ändert sich die UI-Struktur, müssen nur die Driver angepasst werden, nicht hunderte von Tests.

4. Continuous Integration Pipeline

GitHub Actions führt bei jedem Push und Pull Request eine umfassende Test-Pipeline aus. Die CI-Konfiguration in `.github/workflows/ci.yml` definiert mehrere parallel laufende Jobs, die verschiedene Aspekte validieren.

4.1. Job-Struktur und Parallelisierung

Die Pipeline besteht aus fünf Haupt-Jobs:

1. Unit-Tests Job

Läuft auf Ubuntu und Windows mit Matrix-Strategy:

```
strategy:
  fail-fast: false
  matrix:
    os: [ubuntu-latest, windows-latest]
```

Das fail-fast: false stellt sicher, dass alle Matrix-Kombinationen durchlaufen, auch wenn eine fehlschlägt. So erhält man vollständiges Feedback. Der Job führt `go test ./... -short` aus, wobei -short Integrationstests überspringt. Coverage-Daten werden in `/tmp/code_coverage` gesammelt und als Artifact hochgeladen.

2. Integration-Tests Job

Testet gegen vier Git-Versionen parallel:

```
matrix:
  git-version:
    - 2.32.0 # oldest supported
    - 2.38.2 # first with rebase.updateRefs
    - 2.44.0 # recent stable
    - latest # bleeding edge
```

Für Versionen != latest wird Git aus den Quellen kompiliert, was durch Caching optimiert wird. Das Script `run_integration_tests.sh` führt die Integrationstests aus und sammelt Coverage-Daten. Diese Matrix ist essentiell, da Git zwischen Versionen Breaking Changes haben kann.

3. Build Job

Kompiliert Binaries für Linux, Windows und Darwin:

```
- name: Build linux binary
  run: GOOS=linux go build
- name: Build windows binary
  run: GOOS=windows go build
- name: Build darwin binary
  run: GOOS=darwin go build
```

Dies validiert Cross-Platform-Compatibility und stellt sicher, dass der Code auf allen Zielsystemen kompiliert.

4. Check-Codebase Job

Validiert Codebase-Konsistenz:

- Vendor-Directory Match mit `go mod vendor`
- go.mod Cleanness mit `go mod tidy`
- Auto-Generated Files mit `go generate ./...`
- Dateinamen-Konventionen

Diese Checks verhindern, dass Dependencies out-of-sync geraten oder generierter Code veraltet ist.

5. Lint Job

Verwendet golangci-lint v2.4.0 zur statischen Code-Analyse. Der Linter prüft:

- Code-Smell und Anti-Patterns
- Potenzielle Bugs (nil-dereferences, race conditions)
- Performance-Issues (ineffiziente Loops, String-Concatenation)
- Security-Probleme (weak crypto, SQL injection risks)
- Stil-Violations (naming conventions, unused variables)

4.1.1. Coverage-Aggregation

Ein spezieller upload-coverage Job aggregiert Coverage-Daten von allen Test-Jobs:

```
needs: [unit-tests, integration-tests]
```

Er lädt alle Coverage-Artifacts herunter, merged sie mit `go tool covdata` und uploaded das Ergebnis zu Codacy für Tracking und Visualisierung. Dies gibt einen Gesamt-Coverage-Überblick über Unit- und Integrationstests kombiniert.

4.1.2. Fail-Safes und Quality Gates

Die Pipeline implementiert mehrere Safeguards:

- **Fixup-Commits Check:** Verhindert versehentliches Mergen von `fixup!` commits
- **Required Labels:** PRs benötigen spezifische Labels für Kategorisierung
- **No Direct Master Pushes:** Nur via Pull Request möglich
- **All Checks Must Pass:** PRs können nur bei grüner Pipeline gemerget werden

Die Gesamt-Ausführungszeit liegt typischerweise bei 15-20 Minuten dank Parallelisierung. Ohne würde die serielle Ausführung über eine Stunde dauern.

4.2. Code-Coverage-Analyse und Metriken

Die Coverage-Erfassung nutzt Go 1.21+,s neues Coverage-Format, das auch Integration-Test-Coverage von kompilierten Binaries erfassen kann.

4.2.1. Coverage-Erfassung in Unit-Tests

```
go test ./... -short -cover -args "-test.gocoverdir=/tmp/code_coverage"
```

Das `-test.gocoverdir` Flag schreibt Coverage-Daten in ein Verzeichnis statt einer einzelnen Datei, was spätere Aggregation vereinfacht.

4.2.2. Coverage-Erfassung in Integrationstests

Integrationstests kompilieren lazygit mit Coverage-Instrumentation:

```
LAZYGIT_GOCOVERDIR=/tmp/code_coverage go test -cover -coverpkg=github.com/jesseduffield/lazygit/pkg/...
```

Die kompilierte Binary schreibt beim Ausführen Coverage-Daten. Dies ermöglicht Coverage-Tracking für Code, der nur durch UI-Interaktionen erreicht wird.

4.2.3. Coverage-Merging

Nach Testausführung werden Coverage-Daten gemerged:

```
go tool covdata merge -i=/tmp/code_coverage -o=/tmp/code_coverage_merged  
go tool covdata textfmt -i=/tmp/code_coverage_merged -o coverage.out
```

Das resultierende `coverage.out` kann visualisiert werden:

```
go tool cover -html=coverage.out
```

Dies generiert einen HTML-Report mit farbcodierter Darstellung: Grün für getestete, Rot für ungetestete Zeilen.

4.2.4. Coverage-Metriken nach Komponenten

Basierend auf Coverage-Reports zeigt sich folgende Verteilung:

- Command-Builder (pkg/commands/git_commands): 75-85%
- Loader-Komponenten (Branch, Commit, File Loader): 70-80%
- Controller-Logic (pkg/gui/controllers): 60-70%

- Presentation-Layer (pkg/gui/presentation): 40-55%
- UI-Rendering (pkg/gui/views): 25-40%

Die niedrigere UI-Coverage ist typisch für Terminal-Applikationen. UI-Logik ist schwerer zu testen und ändert sich häufiger, was aufwändige Test-Maintenance bedeutet. Das Projekt kompensiert dies durch umfangreiche Integrationstests, die UI-Komponenten indirekt testen.

4.2.5. Testmetriken

Die Testsuite umfasst:

- 80+ Unit-Test-Dateien in pkg/
- 450+ Integrationstests in pkg/integration/tests/
- 2500 einzelne Unit-Test-Funktionen
- Gesamt-Testausführungszeit: 10 Sekunden (Unit), 5-10 Minuten (Integration)
- Test-Code zu Produktionscode-Ratio: 1.2:1 in kritischen Packages

Die schnelle Unit-Test-Ausführung ermöglicht Test-Driven Development mit sofortigem Feedback. Entwickler können `go test ./...` nach jeder Änderung laufen lassen ohne spürbare Verzögerung.

5. Testumgebung und Werkzeuge

5.1. Verwendete Tools

Lazygit nutzt Go's Standard-Testing-Framework mit dem `testing`-Package und `go test`-Runner. Testify ergänzt dies um Assertions wie `assert.Equal()` und `require.NoError()`. Die Go-Git-Bibliothek ermöglicht programmatisches Setup von Test-Repositories. Die PTY-Bibliothek simuliert Terminal-Interaktionen für End-to-End-Tests. Golangci-lint prüft Code-Qualität, codespell findet Rechtschreibfehler, gofmt/goimports formatieren Code automatisch.

5.2. Testumgebungen

Die Test-Matrix deckt verschiedene Konfigurationen ab:

Komponente	Variante
Betriebssysteme	Ubuntu, Windows
Go-Versionen	1.25
Git-Versionen	2.32.0, 2.38.2, 2.44.0, latest

Terminaltypen	xterm-256color, dumb
---------------	----------------------

Version 2.32.0 ist die Minimum-Version, 2.38.2 führte `rebase.updateRefs` ein, latest testet Zukunftskompatibilität. Terminal-Typen validieren, dass lazygit bei fehlenden Features degradiert.

5.3. Docker-Container

Das Projekt bietet Docker-Unterstützung mit Alpine-Linux-basierten Containern. Dev Container-Support ermöglicht Entwicklung ohne lokale Tool-Installation. Die `.devcontainer`-Konfiguration enthält alle Dependencies und IDE-Extensions für sofortige Produktivität.

6. Entwurf eigener Testfälle

6.1. Identifikation von Testlücken

Die Analyse erfolgte systematisch durch Coverage-Metriken aus der CI-Pipeline, manuelle Code-Reviews und Git-Commit-Historie-Analysen. Zwei signifikante Lücken wurden identifiziert: Die Tag-Kommandos in `pkg/commands/git_commands/tag.go` enthielten Funktionen ohne Tests, ebenso die `StringStack`-Datenstruktur in `pkg/utils/string_stack.go`. Die Priorisierung erfolgte nach Risiko: Tag-Operationen sind kritisch und können bei Fehlfunktion zu Problemen in der Versionsverwaltung führen.

6.2. Testfalldesign und Methodik

Für Tag-Kommandos wurde table-driven Testing gewählt. Die Tests für `CreateLightweightObj` decken vier Szenarien ab: Einfacher Tag auf HEAD, Tag auf spezifischem Commit, Force-Flag zum Überschreiben und Kombination aller Parameter. Diese Szenarien validieren die bedingte Logik in `ArgIf`-Konstrukten vollständig.

Annotierte Tags erhielten analoge Tests mit zusätzlichem `msg`-Parameter. `IsTagAnnotated` nutzt Mock-basierte Tests mit verschiedenen Git-Ausgaben inkl. Whitespace-Varianten, um robustes Parsing zu validieren.

`StringStack`-Tests folgten einem zustandsbasierten Ansatz. `TestStringStack_PushAndPop` validiert LIFO-Semantik, `TestStringStack_PopEmptyStack` prüft graceful degradation,

TestStringStack_IsEmpty testet State-Erkennung, TestStringStack_Clear validiert vollständiges Zurücksetzen und TestStringStack_MultipleOperations prüft komplexe Sequenzen.

6.3. Implementierung

Die Implementation folgte Projekt-Konventionen. `tag_test.go` definiert Szenario-Strukturen mit Eingaben und erwarteten Git-Commands. Der FakeRunner simuliert Git ohne Ausführung. Assertions vergleichen konstruierte mit erwarteten Befehlen. StringStack-Tests verwenden klassisches Unit-Test-Pattern ohne Table-Driven-Ansatz, da primär Zustandsübergänge getestet werden.

6.4. Testergebnisse

Alle 18 Tests bestanden beim ersten Durchlauf. Lokale Ausführung mit `go test ./pkg/commands/git_commands -run TestTag -v` dauerte unter 10ms. Die CI-Pipeline validierte auf Ubuntu und Windows ohne Plattform-Probleme.

Die Coverage-Analyse zeigt signifikante Verbesserungen:

Für `tag.go` wurden 5 von 8 Funktionen auf 100% Coverage gebracht:

- NewTagCommands: 0% → 100%
- CreateLightweightObj: 0% → 100%
- CreateAnnotatedObj: 0% → 100%
- LocalDelete: 0% → 100%
- IsTagAnnotated: 0% → 100%

Remote-Operationen (HasTag, Push, ShowAnnotationInfo) blieben bei 0%, da sie Netzwerk erfordern und besser durch Integrationstests abgedeckt werden. Die Gesamt-Coverage von `tag.go` stieg von 0% auf 62.5%.

Für `string_stack.go` erreichten alle vier Funktionen 100% Coverage:

- Push: 0% → 100%
- Pop: 0% → 100%
- IsEmpty: 0% → 100%
- Clear: 0% → 100%

Auf Package-Ebene verbesserte sich `pkg/commands/git_commands` von 37.1% auf 37.6% (+0.5 Prozentpunkte) und `pkg/utls` von 58.2% auf 59.6% (+1.4 Prozentpunkte).

7. Testauswertung und Metriken

Die Coverage-Verbesserungen sind messbar und signifikant. Für `tag.go` stieg die Coverage von 0% auf 62.5%, wobei alle getesteten Funktionen 100% Coverage erreichten. Nur drei Remote-Funktionen blieben ungetestet. `string_stack.go` erreichte vollständige 100% Coverage für alle Funktionen.

Auf Package-Ebene verbesserte sich `pkg/commands/git_commands` um 0.5 Prozentpunkte (37.1% → 37.6%) und `pkg/utls` um 1.4 Prozentpunkte (58.2% → 59.6%). Diese scheinbar kleinen Zahlen sind bedeutsam, da beide Packages umfangreich sind und die neuen Tests gezielt Lücken schließen.

Lazygit verfügt über 80+ Test-Dateien mit 2500 Unit-Test-Funktionen. Unit-Tests laufen in unter einer Minute. Die Test-Code-Ratio ist ausgewogen – kritische Packages haben umfangreichere Tests. Die Tests integrierten sich nahtlos in die CI-Pipeline durch Standard-Go-Patterns und laufen auf allen Plattformen.

7.1. Empfehlungen und Fazit

7.1.1. Bewertung der Teststrategie

Lazygit verfügt über eine ausgereifte Teststrategie mit Unit-Tests, Integrationstests und umfassender CI/CD-Integration. Nach der detaillierten Analyse und praktischen Arbeit am Projekt lassen sich klare Stärken und Verbesserungspotenziale identifizieren.

Stärken der aktuellen Teststrategie sind vielfältig. Die konsequente Verwendung von Mock-Objekten für schnelle, deterministische Tests ermöglicht es, hunderte von Tests in Sekunden auszuführen. Der FakeRunner abstrahiert Git-Operationen effektiv und macht Tests unabhängig von externer Git-Installation. Table-driven Tests sorgen für hohe Wartbarkeit und ermöglichen einfache Erweiterung durch Hinzufügen neuer Szenarien. Die Matrix-Builds in der CI-Pipeline gewährleisten Plattformkompatibilität über Ubuntu und Windows sowie ver-

schiedene Git-Versionen. Die Trennung von Unit-, Integrations- und End-to-End-Tests folgt Best Practices und ermöglicht differenzierte Testausführung.

Die Test-Infrastruktur ist gut durchdacht. Tests sind nahe am Produktionscode lokalisiert, was Wartung erleichtert. Die Verwendung von Go's Standard-Testing-Framework ohne schwere Abhängigkeiten hält das Projekt schlank. Testify ergänzt die Standardbibliothek um lesbare Assertions ohne übermäßige Abstraktion. Die CI-Pipeline ist robust und bietet schnelles Feedback bei Pull Requests.

Schwächen zeigen sich in variierender Coverage zwischen Komponenten. Die Coverage-Analyse ergab, dass kritische Business-Logik wie Command-Builder gut getestet ist (70-90% Coverage), während UI-Code und einige Utility-Funktionen deutlich geringere Coverage aufweisen (20-50%). Dies ist teilweise durch schwierige UI-Testbarkeit bedingt, zeigt aber auch, dass systematische Coverage-Analysen bisher nicht konsequent zur Identifikation von Testlücken genutzt wurden.

Ältere Komponenten haben minimale Tests, was Refactoring-Risiken birgt. Einige Module aus den frühen Entwicklungsphasen des Projekts enthalten komplexe Logik ohne adäquate Test-Abdeckung. Refactorings in diesen Bereichen sind riskant, da keine Tests existieren, die Regressions aufdecken würden. Dies führt zu einer „Test-Debt“, die zukünftige Entwicklung bremst.

Test-Dokumentation ist begrenzt, was Einstiegshürden für neue Mitwirkende schafft. Während der Code selbst gut strukturiert ist, fehlt eine zentrale Dokumentation über Testing-Best-Practices im Projekt. Neue Contributors müssen durch Lesen existierender Tests lernen, wie Tests geschrieben werden sollten. Eine Testing-Guide würde den Onboarding-Prozess erheblich beschleunigen.

Performance-Tests und Benchmarks fehlen weitgehend. Go's Testing-Framework unterstützt Benchmarks nativ, aber lazygit nutzt diese kaum. Für Performance-kritische Operationen wie Git-Log-Parsing oder UI-Rendering wären Benchmarks wertvoll, um Performance-Regressionen frühzeitig zu erkennen.

7.1.2. Handlungsempfehlungen

Basierend auf der Analyse lassen sich folgende konkrete Empfehlungen ableiten:

Systematische Coverage-Analyse etablieren: Integration von Coverage-Tools wie Codecov in die CI-Pipeline mit visualisierten Reports. Mindest-Coverage-Anforderungen für Pull Requests einführen, beispielsweise dass neue Funktionen mindestens 80% Coverage haben müssen. Regelmäßige Coverage-Reviews durchführen, um Lücken zu identifizieren und zu priorisieren. Fokus auf kritische und häufig genutzte Funktionen legen, nicht auf absolute Coverage-Zahlen.

Erweiterung der Test-Dokumentation: Einen zentralen Testing-Guide erstellen nach Vorbild von `TEST_ERKLAERUNG.md`, der verschiedene Test-Patterns erklärt. Best Practices dokumentieren für Unit-Tests, Table-Driven Tests, Mock-Nutzung und Integration-Tests. Beispiele für häufige Test-Szenarien bereitstellen, wie Command-Tests, Parser-Tests und UI-Tests. Contribution-Guidelines um Testing-Abschnitt erweitern, der erklärt, wann welche Art von Tests angebracht ist.

Performance-Benchmarks einführen: Benchmarks für kritische Operationen implementieren, besonders Git-Log-Parsing, Branch/Tag-Listing und UI-Rendering. Diese Benchmarks in CI-Pipeline integrieren und Performance-Regressionen automatisch erkennen. Baseline-Messungen etablieren und bei signifikanten Abweichungen Warnings generieren.

Testlücken systematisch schließen: Ältere Module ohne Tests priorisieren und schrittweise Test-Coverage aufbauen. Beginnend mit den kritischsten Funktionen arbeiten und sich zu weniger kritischen vorarbeiten. Bei jedem Bugfix einen reproduzierenden Test hinzufügen, um zukünftige Regressionen zu verhindern. „Boy Scout Rule“ anwenden: Jeden Code-Bereich, den man anfasst, etwas besser hinterlassen als man ihn vorgefunden hat.

Test-Code-Reviews institutionalisieren: Regelmäßige Reviews von Test-Code durchführen, nicht nur Produktionscode. Auf Test-Qualität achten: Sind Tests verständlich? Testen sie das richtige? Sind sie wartbar? Test-Antipatterns identifizieren und eliminieren, wie übermäßiges Mocking, fragile Tests oder Tests die Implementierungsdetails testen.

Automatisierung ausbauen: Pre-commit Hooks einführen, die Tests lokal ausführen bevor Code gepusht wird. Automatische Test-Generierung evaluieren für einfache Fälle wie Getter/Setter oder simple Data-Transformationen. Mutation-Testing ausprobieren, um Qualität existierender Tests zu validieren.

7.1.3. Fazit

Diese Arbeit analysierte das Testkonzept von lazygit umfassend und erweiterte es durch eigene Testfälle. Lazygit verfügt über eine ausgereifte Teststrategie mit Unit-Tests, Table-Driven Tests, Integrationstests und robuster CI/CD-Pipeline, die als Vorbild für andere Go-Projekte dienen kann.

Die durchgeführten Arbeiten umfassten mehrere Phasen: Die initiale Analyse identifizierte Testlücken in Tag-Kommandos und StringStack durch Coverage-Analysen und Code-Reviews. Die Implementierung umfasste 18 Testfälle mit 13 Sub-Tests für Tags und fünf für Stack-Operationen, alle im table-driven bzw. zustandsbasierten Test-Stil. Die Erstellung umfassender Dokumentation in `TEST_ERKLAERUNG.md` bietet didaktisches Material für neue Contributors. Alle Tests wurden erfolgreich in die CI-Pipeline integriert und bestehen auf allen Plattformen.

Die theoretischen Grundlagen des Software-Testens wurden praktisch angewendet und validiert. Table-driven Tests demonstrieren Go-Best-Practices für maximale Testabdeckung mit minimalem Code-Overhead. Mock-basiertes Testing zeigt, wie Unit-Tests schnell und deterministisch gestaltet werden können. Die Kombination von Unit- und Integrationstests folgt dem Pyramiden-Modell und optimiert die Balance zwischen Geschwindigkeit und Gründlichkeit.

Lazygit dient als exzellentes Beispiel für durchdachte Teststrategien in Open-Source-Projekten. Die konsequente Anwendung von Testing-Best-Practices trägt zur hohen Code-Qualität bei und ermöglicht schnelle, konfidente Entwicklung. Die erstellten Tests und Dokumentationen verbessern die Codequalität nachhaltig und erleichtern zukünftigen Mitwirkenden den Einstieg.

Persönlich war diese Arbeit lehrreich in mehrfacher Hinsicht. Die praktische Arbeit an einem realen Open-Source-Projekt vermittelte Einblicke, die durch rein akademische Übungen

nicht möglich wären. Die Herausforderung, Tests für existierenden Code zu schreiben, unterscheidet sich fundamental von Test-First-Ansätzen und erfordert sorgfältige Analyse. Die Notwendigkeit, Projekt-Konventionen zu folgen und sich in bestehende Code-Bases einzuarbeiten, spiegelt realistische Berufspraxis wider.

Die Erkenntnisse dieser Arbeit sind über lazygit hinaus wertvoll. Table-driven Tests sind in jedem Go-Projekt anwendbar. Mock-basierte Unit-Tests sind sprachübergreifend relevant. Die CI/CD-Patterns mit GitHub Actions lassen sich auf andere Projekte übertragen. Und die systematische Identifikation von Testlücken ist eine Fähigkeit, die in jeder professionellen Software-Entwicklung benötigt wird.

Zukünftige Arbeiten könnten diese Analyse erweitern durch Performance-Benchmarking kritischer Komponenten, Mutation-Testing zur Validierung der Test-Qualität, End-to-End-Test-Automatisierung für komplexere User-Journeys oder Fuzz-Testing für Parser und Input-Validierung. Lazygit bietet ein reichhaltiges Umfeld für weitere Experimente im Software-Testing.

Quellen

- [1] E. W. Dijkstra, „The Humble Programmer“, *ACM Turing Award Lectures*. Association for Computing Machinery, New York, NY, USA, 2007. doi: 10.1145/1283920.1283927.
- [2] P. Liggesmeyer, *Software-Qualität: Testen, Analysieren und Verifizieren von Software*, 2. Auflage. Spektrum Akademischer Verlag, 2009.
- [3] D. W. Hoffmann, *Software-Qualität*, 2. Auflage. Springer Vieweg, 2013.
- [4] The Go Authors, „Table Driven Tests“. Zugriffen: 15. Januar 2026. [Online]. Verfügbar unter: <https://github.com/golang/go/wiki/TableDrivenTests>